



Universidad Nacional
Federico Villarreal

Sistema de Alerta Temprana de Fatiga y Somnolencia Multimodal para Choferes Basado en Edge AI y Protocolos de Comunicación

Integrantes:

- Ruiz Meza Ricardo Antonio
- Vizcardo Zegarra Roberto Carlos
- Picon Ayala Yan Levi

Docente

- Villlafuerte Hernan

Sección: “C”

Año : 2026



Índice de Contenidos

01 Arquitectura del
Microcontrolador y Mapa de
Pines

02 Gestión de Memoria y Manejo
de Búfers

03 Diagrama de Flujo - Fase 1:
Inicialización

04 Diagrama de Flujo - Fases 2 y 3

05 Diagrama de Flujo - Fase 4

06 Código Fuente Parcial

ARQUITECTURA DEL MICROCONTROLADOR Y MAPA DE PINES

Procesador Central

*Chip Tensilica Xtensa de 32 bits Dual-Core a 240 MHz
(ESP32-CAM)*

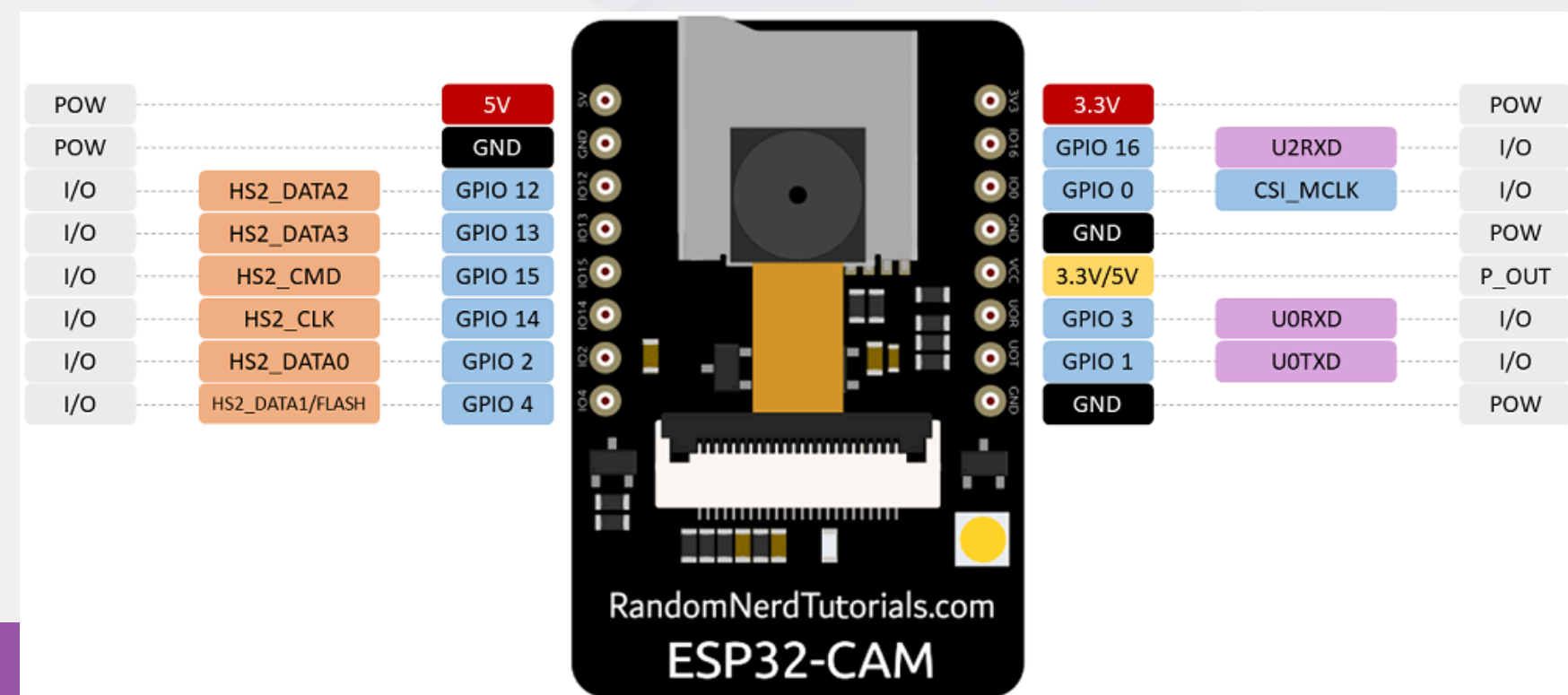
Procesamiento Asíncrono

*Un núcleo se dedica exclusivamente a sostener la
pila de red Wi-Fi y el streaming, mientras el segundo
núcleo atiende las solicitudes de alerta del buzzer sin
congelar el video.*

Mapa de Pines (Hardware Base)

- **Captura Óptica:** Sensor de video OV2640 y diodo LED Infrarrojo.
- **Conexión de Programación:** Pines RX/TX enlazados al adaptador serial FTDI de color rojo (interfaz UART a 115,200 baudios).
- **Actuador Físico:** Pin digital GPIO 2 configurado como salida para el buzzer de 5V.

```
1 #include "esp_camera.h"
2 #include <WiFi.h>
3 #include "esp_http_server.h"
4
5 // =====
6 // 1. PINES (AI-THINKER) - MAPEO DE HARDWARE BASE
7 // =====
8
9 #define PWDN_GPIO_NUM    32
10 #define RESET_GPIO_NUM  -1
11 #define XCLK_GPIO_NUM    0
12 #define SIOD_GPIO_NUM    26
13 #define SIOC_GPIO_NUM    27
14 #define Y9_GPIO_NUM      35
15 #define Y8_GPIO_NUM      34
16 #define Y7_GPIO_NUM      39
17 #define Y6_GPIO_NUM      36
18 #define Y5_GPIO_NUM      21
19 #define Y4_GPIO_NUM      19
20 #define Y3_GPIO_NUM      18
21 #define Y2_GPIO_NUM       5
22 #define VSYNC_GPIO_NUM   25
23 #define HREF_GPIO_NUM    23
24 #define PCLK_GPIO_NUM    22
25 #define BUZZER_PIN        2           // Pin GPIO 2 enlazado al buzzer de 5V
```



GESTIÓN DE MEMORIA Y MANEJO DE BÚFERS

Limitación de Hardware

Los 520 KB de la memoria SRAM interna son insuficientes para procesar video y causarían un colapso del sistema (Crash)

Estrategia de Diseño (Búfer Masivo)

Un núcleo se dedica exclusivamente a sostener la pila de red Wi-Fi y el streaming, mientras el segundo núcleo atiende las solicitudes de alerta del buzzer sin congelar el video.

Flujo de Datos a Nivel de Búfer

La función nativa esp_camera_fb_get() captura la foto desde el lente OV2640 en resolución ligera QVGA (320x240) y aloja los bytes en formato JPEG temporalmente en la PSRAM.

Protección del Sistema

Al liberar la memoria interna, la SRAM queda disponible únicamente para las variables globales, contraseñas de red y las pilas de trabajo del servidor web.

Componente de Memoria	Capacidad Física	Función Principal en el Proyecto
Memoria SRAM Interna	520 KB	Guarda variables globales, contraseñas de red y controla los hilos del servidor web.
Memoria PSRAM Externa	4 MB	Actúa como el buffer masivo donde se guardan temporalmente las fotos de la cámara antes de transmitirse.

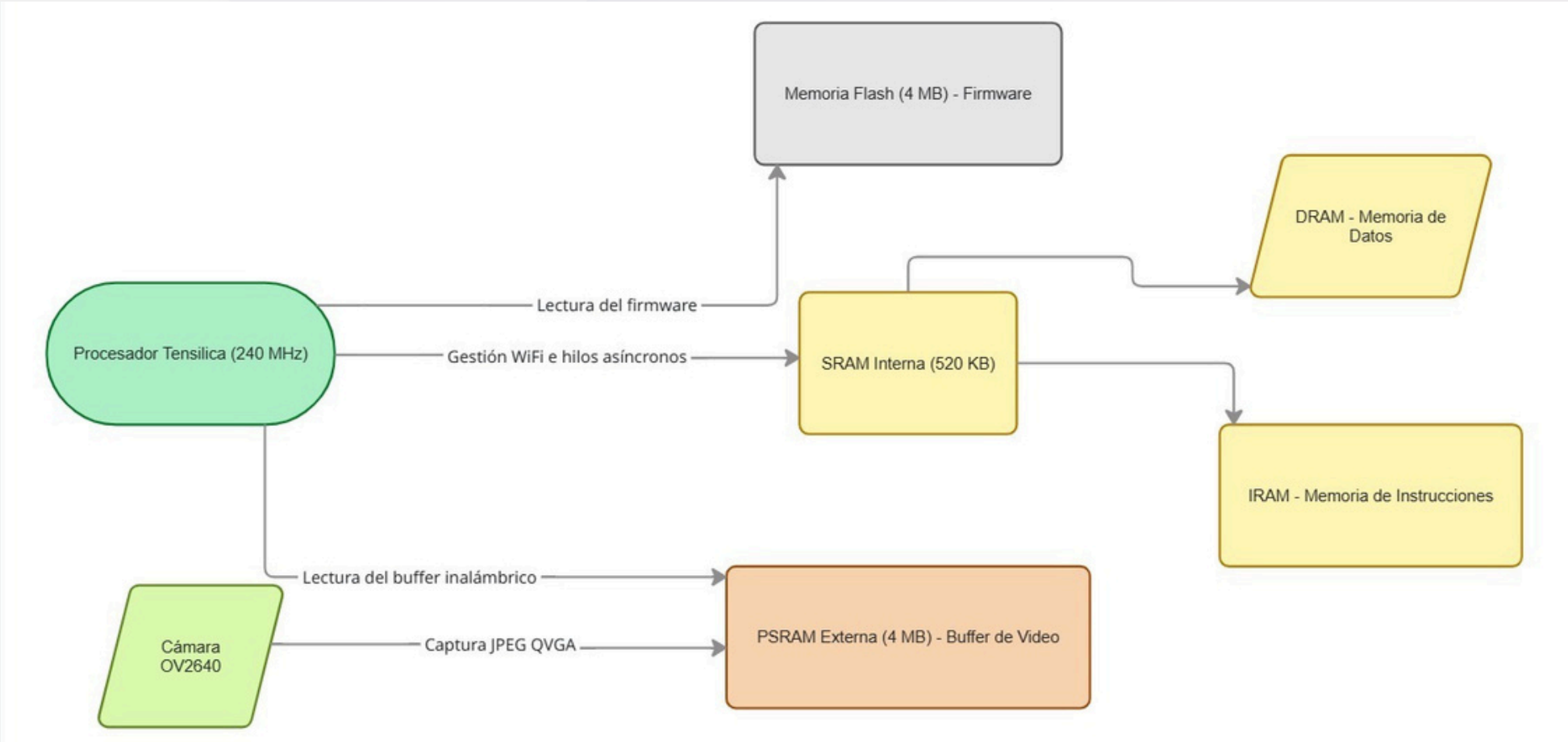
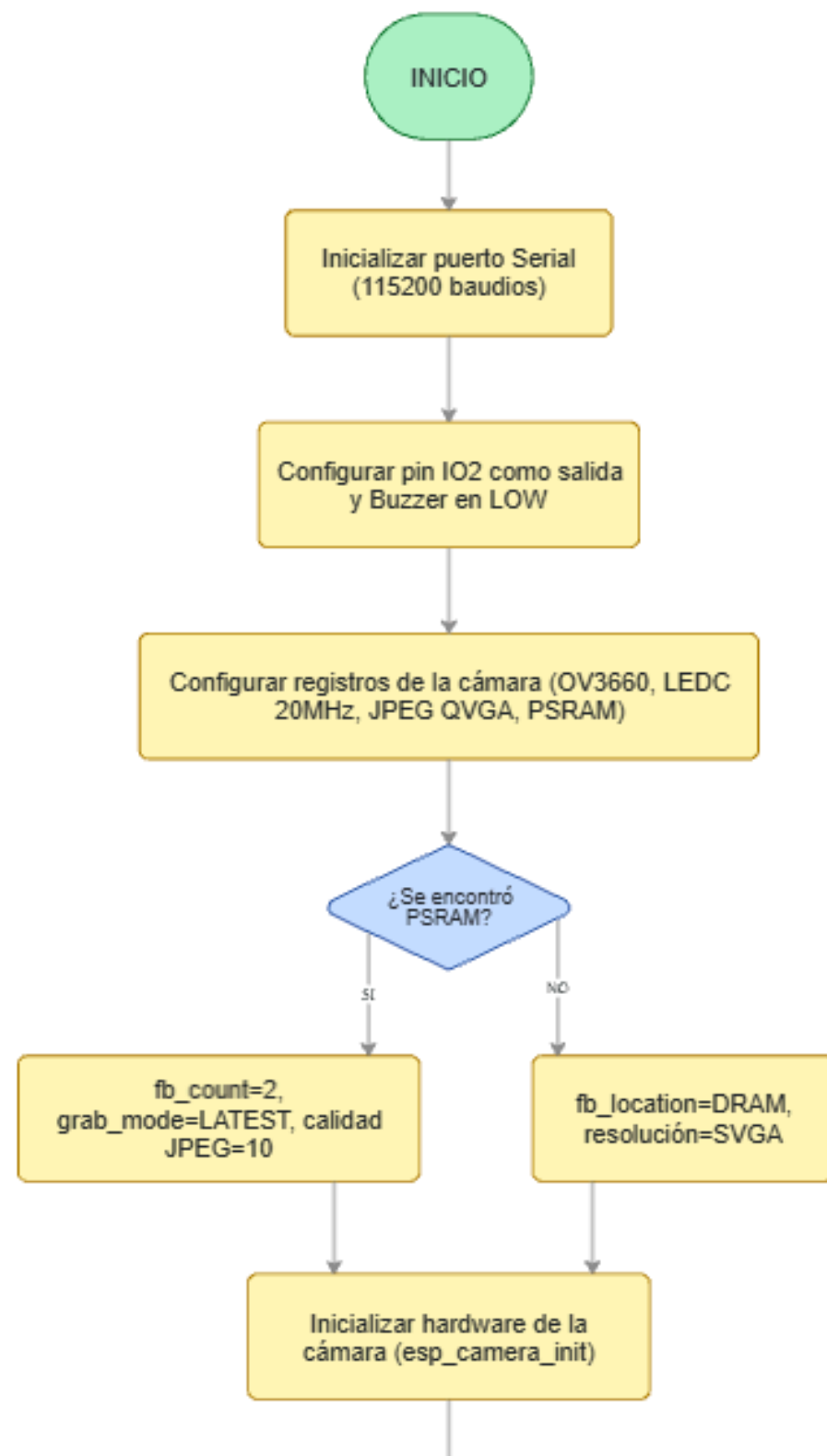
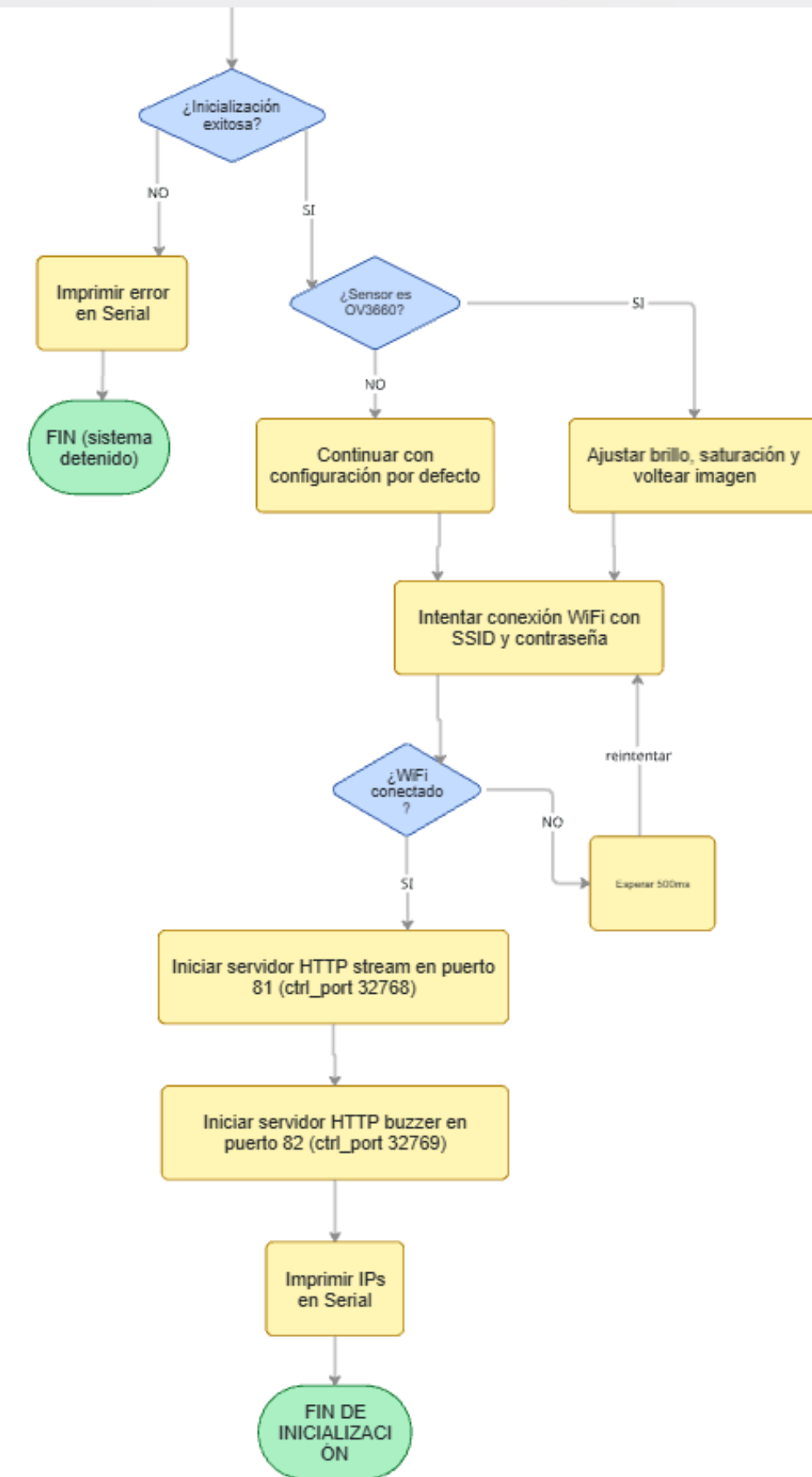


DIAGRAMA DE FLUJO – FASE 1: INICIALIZACIÓN



La tarjeta prende y activa la comunicación por cable con la computadora a una velocidad de 115,200 baudios para avisar que todo está bien. En ese mismo instante, se separa un espacio en la memoria PSRAM externa de 4 MB para guardar las fotos que tome la cámara, evitando que la memoria interna se llene y la tarjeta se apague.

Se arranca el sensor de la cámara (OV2640). El chip revisa que el lente esté bien conectado y ajusta automáticamente el brillo y contraste para que se pueda ver con claridad incluso en la oscuridad de la noche usando el LED infrarrojo

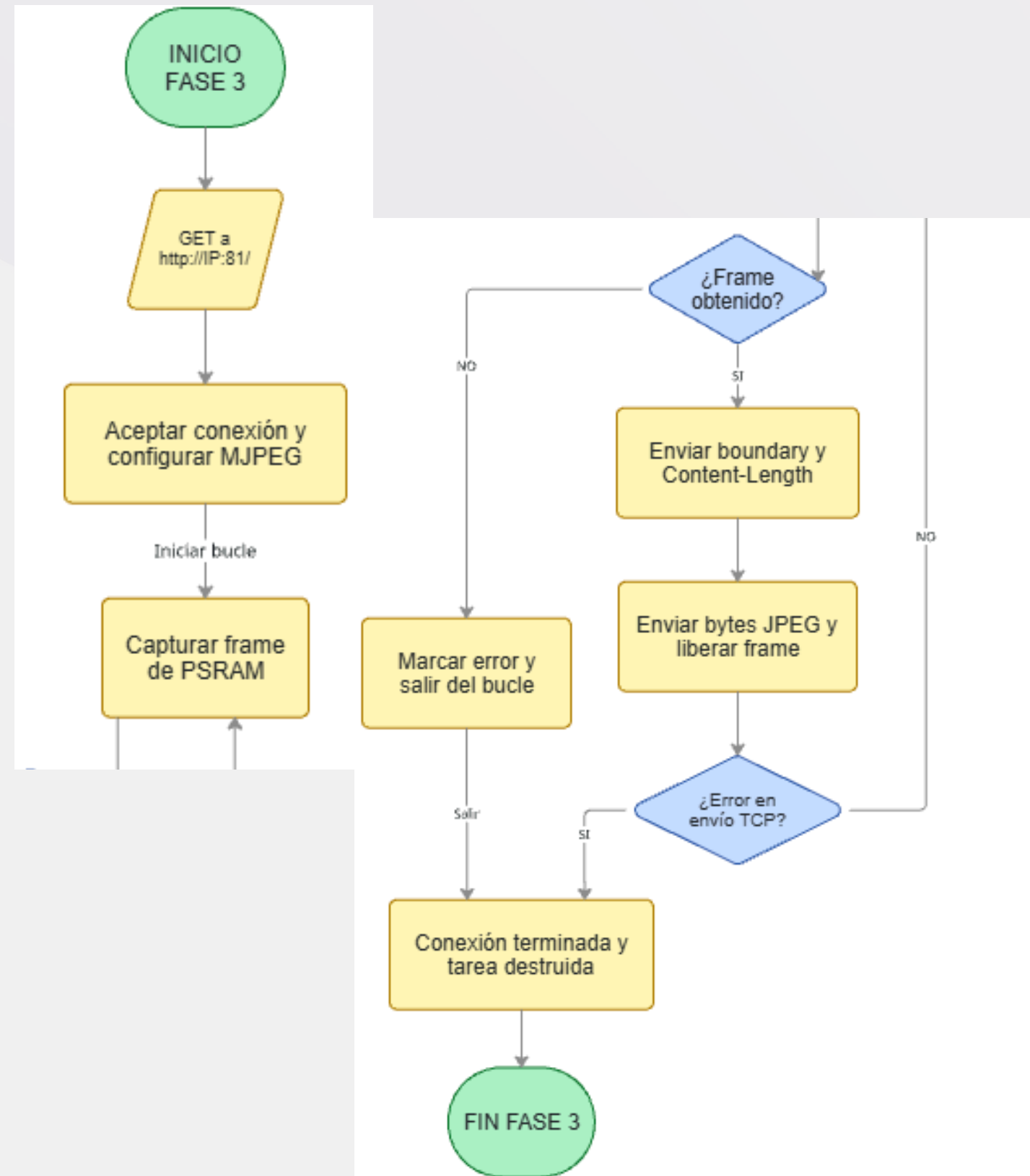
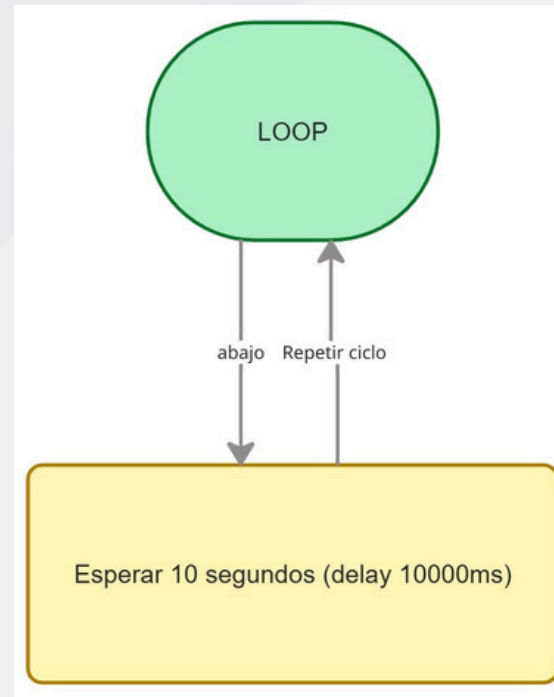


La tarjeta enciende su antena y se conecta a la red Wi-Fi local instalada en el vehículo. Al conectarse, crea dos canales (puertos) independientes: el Puerto 81 que se encargará exclusivamente de transmitir el video y el Puerto 82 que se quedará esperando la orden para activar el buzzer.

DIAGRAMA DE FLUJO – FASES 2 Y 3

Fase 2

El void loop() se mantiene libre ejecutando solo un retardo de bajo consumo (delay(10000ms)). El kernel de FreeRTOS asume el control asíncrono total



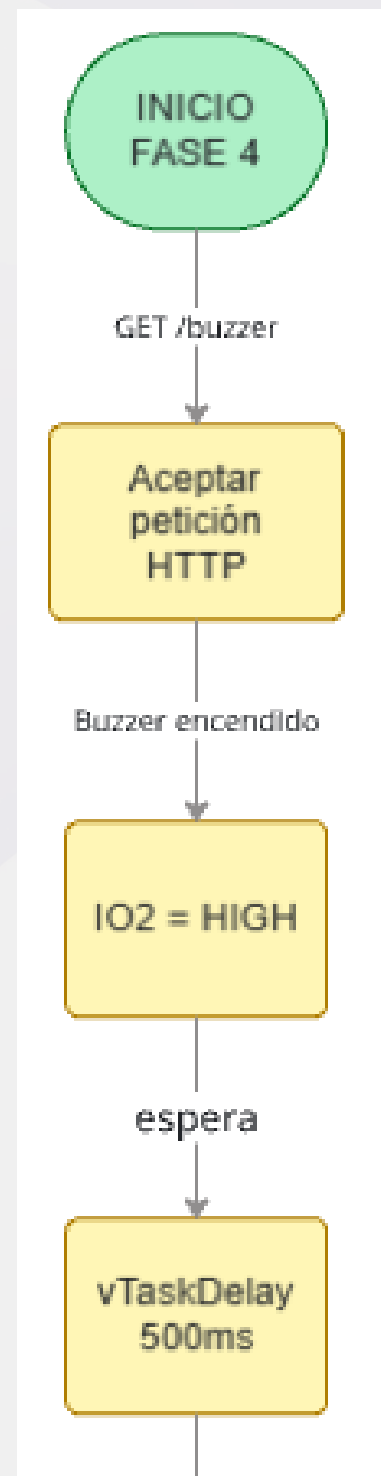
Se activa cuando Python realiza una petición HTTP GET al puerto 81 de la tarjeta. El servidor acepta la conexión e inicia una transmisión continua bajo el protocolo Multipart MJPEG.

En cada ciclo, la función nativa `esp_camera_fb_get()` captura el fotograma JPEG almacenado temporalmente en la PSRAM externa.

Tras enviar la imagen por la red local, se ejecuta `esp_camera_fb_return()` para vaciar el búfer inmediatamente y evitar el colapso (Overflow) del chip.

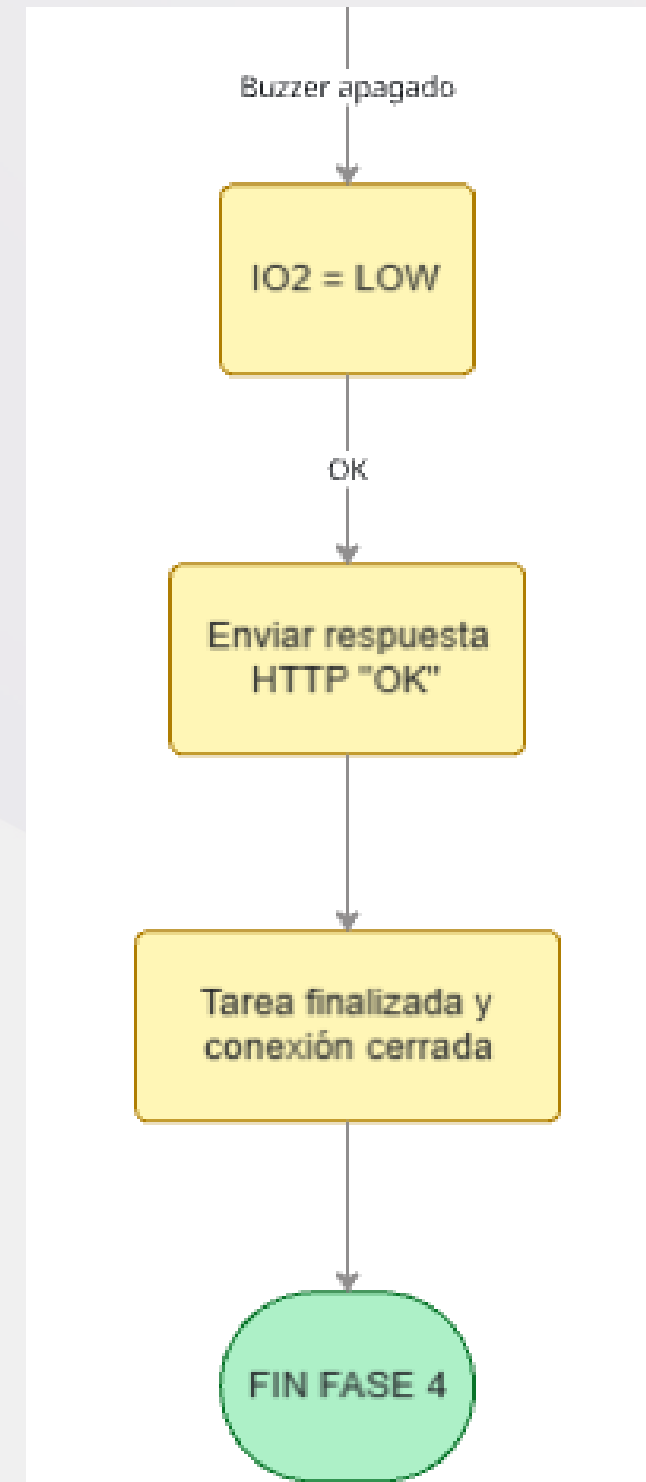
Ante cualquier error de lectura o desconexión TCP, el lazo se rompe drásticamente y el socket se cierra para proteger la CPU.

DIAGRAMA DE FLUJO – FASE 4



El puerto 82 corre en paralelo y de forma independiente al video. Espera de manera pasiva la orden de emergencia desde la computadora.

Si Python detecta los ojos cerrados por más de 2 segundos, un hilo secundario envía un HTTP GET rápido a la ruta /buzzer



Al recibir la orden, la rutina `buzzer_handler` conmuta el pin GPIO 2 a HIGH, encendiendo físicamente el buzzer de 5V de inmediato

El kernel de FreeRTOS sostiene el sonido por exactamente 500 milisegundos usando `vTaskDelay()`. Esto evita que el resto de las tareas de red o el flujo de video se congelen. Luego, el pin vuelve a LOW y el puerto retorna a la escucha pasiva.

CÓDIGO FUENTE PARCIAL

```
1  #include "esp_camera.h"
2  #include <WiFi.h>
3  #include "esp_http_server.h"
4
5  // =====
6  // 1. PINES (AI-THINKER) - MAPEO DE HARDWARE BASE
7  // =====
8
9  #define PWDN_GPIO_NUM    32
10 #define RESET_GPIO_NUM  -1
11 #define XCLK_GPIO_NUM    0
12 #define SIOD_GPIO_NUM    26
13 #define SIOC_GPIO_NUM    27
14 #define Y9_GPIO_NUM      35
15 #define Y8_GPIO_NUM      34
16 #define Y7_GPIO_NUM      39
17 #define Y6_GPIO_NUM      36
18 #define Y5_GPIO_NUM      21
19 #define Y4_GPIO_NUM      19
20 #define Y3_GPIO_NUM      18
21 #define Y2_GPIO_NUM       5
22 #define VSYNC_GPIO_NUM    25
23 #define HREF_GPIO_NUM     23
24 #define PCLK_GPIO_NUM     22
25 #define BUZZER_PIN        2          // Pin GPIO 2 enlazado al buzzer de 5V
26
27 const char* ssid      = "MOVISTAR_CF60";
28 const char* password = "Fy5F7BFKxNAfdghzNTvt";
29
30 httpd_handle_t stream_httpd = NULL;
31 httpd_handle_t buzzer_httpd = NULL;
32
```

```
33 // =====
34 // 2. FASE 3: STREAM HANDLER (PUERTO 81)
35 // =====
36
37 #define PART_BOUNDARY "1234567890000000000000987654321"
38 static const char* STREAM_CONTENT_TYPE = "multipart/x-mixed-replace;boundary=" PART_BOUNDARY;
39 static const char* STREAM_BOUNDARY    = "\r\n--" PART_BOUNDARY "\r\n";
40 static const char* STREAM_PART        = "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";
41
42 static esp_err_t stream_handler(httpd_req_t *req) {
43     camera_fb_t* fb = NULL;
44     esp_err_t res = ESP_OK;
45     size_t _jpg_buf_len = 0;
46     uint8_t* _jpg_buf = NULL;
47     char part_buf[64];
48
49     res = httpd_resp_set_type(req, STREAM_CONTENT_TYPE);
50     if (res != ESP_OK) return res;
51     httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "");
52
53     // Bucle infinito condicionado a la actividad del socket TCP inalámbrico
54
55     while (true) {
56         fb = esp_camera_fb_get(); // Captura el fotograma desde la memoria PSRAM
57         if (!fb) {
58             res = ESP_FAIL;
59         } else {
60             _jpg_buf_len = fb->len;
61             _jpg_buf = fb->buf;
62         }
63
64         // Envío secuencial de delimitadores, cabeceras y bytes binarios del JPEG
65
66         if (res == ESP_OK) res = httpd_resp_send_chunk(req, STREAM_BOUNDARY, strlen(STREAM_BOUNDARY));
67         if (res == ESP_OK) {
68             size_t hlen = snprintf(part_buf, 64, STREAM_PART, _jpg_buf_len);
69             res = httpd_resp_send_chunk(req, part_buf, hlen);
70         }
71         if (res == ESP_OK) res = httpd_resp_send_chunk(req, (const char*)_jpg_buf, _jpg_buf_len);
72
73         if (fb) {
74             esp_camera_fb_return(fb);          // Libera de inmediato el búfer para evitar colapsos
75             fb = NULL;
76             _jpg_buf = NULL;
77         }
78         if (res != ESP_OK) break;              // Rompe el bucle si detecta desconexión TCP
79     }
80     return res;
81 }
```

```
83 // =====
84 // 3. FASE 4: BUZZER HANDLER (PUERTO 82)
85 // =====
86 static esp_err_t buzzer_handler(httpd_req_t *req) {
87     digitalWrite(BUZZER_PIN, HIGH);          // Satura eléctricamente el pin GPIO 2 (Enciende)
88     vTaskDelay(500 / portTICK_PERIOD_MS);    // Retardo asíncrono en FreeRTOS por 500 ms
89     digitalWrite(BUZZER_PIN, LOW);           // Corta la energía del pin (Apaga)
90     httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "");
91     httpd_resp_send(req, "OK", 2);           // Envía confirmación HTTP al cliente Python
92     return ESP_OK;
93 }
94
95 // =====
96 // 4. BLOQUE 3: INICIO DE SERVIDORES HTTP
97 // =====
98 void startCameraServer() {
99     httpd_config_t config1 = HTTPD_DEFAULT_CONFIG();
100     config1.server_port = 81; // Puerto multimedia dedicado al streaming de video
101     config1.ctrl_port   = 32768;
102
103     httpd_uri_t stream_uri = {
104         .uri       = "/",
105         .method    = HTTP_GET,
106         .handler    = stream_handler,
107         .user_ctx  = NULL
108     };
109
110     if (httpd_start(&stream_httpd, &config1) == ESP_OK) {
111         httpd_register_uri_handler(stream_httpd, &stream_uri);
112         Serial.println("Servidor stream OK en puerto 81");
113     } else {
114         Serial.println("ERROR: Fallo servidor stream puerto 81");
115     }
116
117     httpd_config_t config2 = HTTPD_DEFAULT_CONFIG();
118     config2.server_port = 82; // Puerto de control dedicado al actuador físico
119     config2.ctrl_port   = 32769;
120
121     httpd_uri_t buzzer_uri = {
122         .uri       = "/buzzer",
123         .method    = HTTP_GET,
124         .handler    = buzzer_handler,
125         .user_ctx  = NULL
126     };
127
128     if (httpd_start(&buzzer_httpd, &config2) == ESP_OK) {
129         httpd_register_uri_handler(buzzer_httpd, &buzzer_uri);
130         Serial.println("Servidor buzzer OK en puerto 82");
131     } else {
132         Serial.println("ERROR: Fallo servidor buzzer puerto 82");
133     }
134 }
```

CÓDIGO FUENTE PARCIAL

```
136 // =====
137 // 5. BLOQUES 1 Y 2: SETUP E INICIALIZACIÓN
138 // =====
139 void setup() {
140     Serial.begin(115200); // Inicializa comunicación serial UART
141     Serial.setDebugOutput(true);
142
143     pinMode(BUZZER_PIN, OUTPUT); // Configura pin del actuador como salida digital
144     digitalWrite(BUZZER_PIN, LOW); // Asegura que la alarma inicie apagada
145
146     // Asignación de registros y estructuras para el hardware de la cámara
147     camera_config_t config;
148     config.ledc_channel = LEDC_CHANNEL_0;
149     config.ledc_timer = LEDC_TIMER_0;
150     config.pin_d0 = Y2_GPIO_NUM;
151     config.pin_d1 = Y3_GPIO_NUM;
152     config.pin_d2 = Y4_GPIO_NUM;
153     config.pin_d3 = Y5_GPIO_NUM;
154     config.pin_d4 = Y6_GPIO_NUM;
155     config.pin_d5 = Y7_GPIO_NUM;
156     config.pin_d6 = Y8_GPIO_NUM;
157     config.pin_d7 = Y9_GPIO_NUM;
158     config.pin_xclk = XCLK_GPIO_NUM;
159     config.pin_pclk = PCLK_GPIO_NUM;
160     config.pin_vsync = VSYNC_GPIO_NUM;
161     config.pin_href = HREF_GPIO_NUM;
162     config.pin_sccb_sda = SIOD_GPIO_NUM;
163     config.pin_sccb_scl = SIOC_GPIO_NUM;
164     config.pin_pwdn = PWDN_GPIO_NUM;
165     config.pin_reset = RESET_GPIO_NUM;
166     config.xclk_freq_hz = 20000000;
167     config.pixel_format = PIXFORMAT_JPEG;
168     config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
169     config.fb_location = CAMERA_FB_IN_PSRAM; // Reserva buffer en memoria externa
170     config.frame_size = FRAMESIZE_QVGA; // Resolución ligera de 320x240 píxeles
171     config.jpeg_quality = 12;
172     config.fb_count = 1;
173
174     // Validación dinámica de la presencia de memoria RAM pseudo-estática (PSRAM)
175     if (psramFound()) {
176         config.jpeg_quality = 10;
177         config.fb_count = 2; // Doble buffer para paralelismo de red
178         config.grab_mode = CAMERA_GRAB_LATEST;
179     } else {
180         config.frame_size = FRAMESIZE_SVGA;
181         config.fb_location = CAMERA_FB_IN_DRAM;
182     }
183
184     esp_err_t err = esp_camera_init(&config);
185     if (err != ESP_OK) {
186         Serial.printf("ERROR cámara: 0x%x\n", err);
187         return;
188     }
189 }
```

```
190 // Calibración y modificación de registros internos del lente OV2640
191 sensor_t* s = esp_camera_sensor_get();
192 if (s != NULL) {
193     if (s->id.PID == OV2640_PID) {
194         s->set_vflip(s, 0);
195         s->set_brightness(s, 1); // Ajuste óptico para entornos nocturnos oscuros
196         s->set_saturation(s, 0);
197     }
198     s->set_framesize(s, FRAMESIZE_QVGA);
199 }
200
201 WiFi.begin(ssid, password); // Conexión inalámbrica a la red local
202 WiFi.setSleep(false);
203 while (WiFi.status() != WL_CONNECTED) {
204     delay(500);
205     Serial.print(".");
206 }
207
208 startCameraServer(); // Despliegue de los sockets HTTP
209
210 Serial.println("\n📡 Servidor activo");
211 Serial.print("Stream : http://");
212 Serial.print(WiFi.localIP());
213 Serial.println(":81/");
214 Serial.print("Buzzer : http://");
215 Serial.print(WiFi.localIP());
216 Serial.println(":82/buzzer");
217 }
218
219 // =====
220 // 6. FASE 2: LOOP PRINCIPAL / IDLE STATE
221 // =====
222 void loop() {
223     // El lazo principal reposa de forma indefinida delegando las tareas a FreeRTOS
224     delay(1000);
225 }
```

**MUCHAS
GRACIAS**